Rapport - SAE 4.ROM.01 - Portfolio

Daris Mbau - Matteo Vargiu - BUT réseaux et télécommunications – RT21 - ROM - FA

SAE 4.ROM.01 - Déployer une infrastructure opérateur et ses services client

Enseignants : M. Abdelhafid ABOUAISSA

Année universitaire : 2024-2025

Sommaire

Sommaire
Introduction
Déploiement de l'infrastructure réseau
- Cahier des charges et besoins
- Présentation de la maquette réseau - Topologie et architecture du réseau
- Plan d'adressage IP et VLAN
- Configuration des équipements réseau
- Mise en place de la sécurité
- Automatisation de l'infrastructure
- Résultats de l'infrastructure réseau

Déploiement des services réseaux dockerisés
- 1. Service VOD :
- 2. Service IPTV :
- 3. Service Web :
- 4. Service VoIP :

Gestion de projet
Conclusion

Introduction

Dans ce rapport, je présente le projet SAE4.ROM.01 - Déployer une infrastructure opérateur et ses services clients . Ce travail, réalisé en binôme : **Daris Mbau** (responsable du déploiement de l'infrastructure réseau) et **Matteo Vargiu** (responsable du déploiement des services réseaux dockerisés), consiste à mettre en place tout le réseau nécessaire pour que des clients puissent accéder à différents services multimédias (VoIP, VOD, IPTV, Web). Ces services sont hébergés dans des conteneurs Docker sur une machine virtuelle.

L'objectif principal est d'assurer que chaque client puisse se connecter facilement et sans interruption à la machine virtuelle qui héberge ces services. Cela demande de bien configurer le routage, y compris le routage multicast pour l'IPTV, ainsi que les protocoles essentiels pour que les flux passent bien, que ce soit en unicast ou en multicast. Il faut aussi gérer les sous-réseaux et préparer l'infrastructure pour garantir la diffusion continue des contenus multimédias.

Ce projet se rapproche d'une vraie mission d'opérateur réseau. Il mélange l'aspect applicatif (avec les services déployés sous Docker) et l'aspect réseau (infrastructure, routage, diffusion). Tout cela est fait dans un environnement simulé mais très proche de la réalité, pour apprendre à proposer des solutions fiables, sécurisées et adaptées aux besoins des clients.

Je vous présente ci-après les captures d'écran et les explications des différentes réalisations, la mise en place et les étapes accomplies jusqu'à la finalisation.

Déploiement de l’infrastructure réseau

Cahier des charges et besoins

1. **Besoins fonctionnels**

- **Fournir un accès réseau à différents clients** qui peuvent être situés sur des sites distants, tout en assurant une bonne qualité de service pour tous.
- **Permettre l’accès à quatre services principaux**, chacun étant déployé dans un conteneur Docker sur la VM : VoIP (téléphonie sur IP), VoD (vidéo à la demande), IPTV (télévision sur IP, flux multicast), Serveur Web / partage de données (Apache, vidéos, etc.)
- **Connecter les clients à ces services**, peu importe leur position sur le réseau (ex : Client 1 et Client 2 sur le schéma), en garantissant qu’ils puissent s’appeler entre eux et accéder à tous les services.

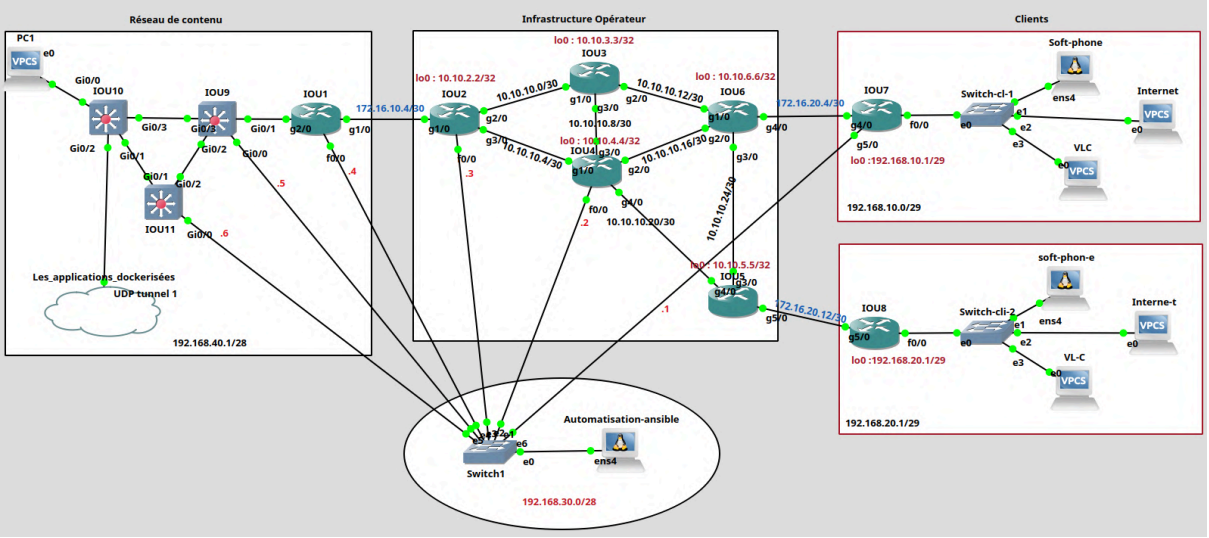
2. **Besoins techniques**

- **Routage avancé** : Configuration du routage entre plusieurs sous-réseaux pour relier tous les équipements.
  Et gestion du routage multicast (PIM-SM, IGMP) pour le service IPTV, afin de permettre une diffusion efficace de la vidéo à plusieurs clients.
- **Gestion des VLANs** pour séparer les différents types de trafic et sécuriser les flux entre les clients et les services.
- **Sécurisation des communications** :
  - Mise en place d’un VPN (ex : DMVPN ou IPsec) pour assurer la confidentialité et la sécurité des échanges entre sites ou entre clients et serveurs.
- **Automatisation du déploiement** : Utilisation d’outils comme Ansible pour automatiser la configuration réseau et le déploiement des services Docker.

3. **Contraintes**

- **Aucune coupure de service** : les flux multimédias doivent être transmis de façon continue et sans interruption, même lors des changements de configuration ou des mises à jour.
- **Pas de NAT** pour ne pas perturber les flux VoIP.
- **Compatibilité avec des protocoles standards** : OSPF pour le routage unicast, PIM-SM pour le multicast, IGMP pour la gestion des groupes multicast.

Présentation de la maquette réseau - Topologie et architecture du réseau



La maquette réseau est composée de trois zones :

- **Réseau de contenu** : une VM héberge les applications (VoIP, VOD, IPTV, Web) dans des conteneurs Docker, reliée à des routeurs Cisco.
- **Infrastructure opérateur** : plusieurs routeurs assurent le transit, la sécurité (VPN) et la diffusion multicast.
- **Réseaux clients** : deux zones clientes, chacune avec un switch et deux terminaux (softphone et lecteur VLC).

Le schéma montre comment chaque partie est reliée pour permettre aux clients d'accéder aux services hébergés, avec un routage optimisé pour les flux multimédias.

- **Routeurs** : gèrent le routage (OSPF pour l'unicast, PIM-SM/IGMP pour le multicast) et la sécurité (VPN).
- **Switchs** : relient les équipements sur chaque zone locale.
- **Machines virtuelles** :
  - Une VM héberge les services dans des conteneurs Docker.
  - Une VM est dédiée à l'automatisation (Ansible).
- **Clients** : chaque site client a accès à tous les services via un softphone et un lecteur multimédia.

L'adressage IP est segmenté pour chaque zone, garantissant la séparation des flux et la sécurité.

Plan d'adressage IP et VLAN

L'adressage est conçu pour être optimal, sans gaspillage d'adresses, et pour séparer clairement chaque zone du réseau : clients, opérateur, services, et administration.

1. **Réseaux clients**

Client 1

- Loopback : `192.168.10.1/29`
- Réseau interne (DHCP) : `192.168.10.0/29`

Client 2

- Loopback : `192.168.20.1/29`
- Réseau interne (DHCP) : `192.168.20.0/29`

Le serveur DHCP est activé sur les routeurs IOU7 et IOU8 pour distribuer ces adresses automatiquement aux équipements des clients.

2. **Infrastructure opérateur**

Loopbacks des routeurs :

- IOU2 : `10.10.2.2/32`
- IOU3 : `10.10.3.3/32`
- IOU4 : `10.10.4.4/32`
- IOU5 : `10.10.5.5/32`
- IOU6 : `10.10.6.6/32`

Liens inter-routeurs :

- IOU2 – IOU3 : `10.10.10.0/30`
- IOU2 – IOU4 : `10.10.10.4/30`
- IOU3 – IOU4 : `10.10.10.8/30`
- IOU3 – IOU6 : `10.10.10.12/30`
- IOU4 – IOU6 : `10.10.10.16/30`
- IOU4 – IOU5 : `10.10.10.20/30`
- IOU5 – IOU6 : `10.10.10.24/30`

3. **Multicast**

Pour les flux IPTV (multicast), j'ai choisi la plage :

- Réseau multicast : `230.0.0.0/29` (classe D)
  - HostMin: `230.0.0.1`
  - HostMax: `230.0.0.6`

4. **Réseau d'administration (Ansible)**

- Switch1 et liens d'administration : `192.168.30.0/28`
- Ce réseau relie Switch1 à IOU8, IOU5, IOU2, IOU1, IOU9, IOU11 ainsi qu'à la VM d'automatisation (Ansible).

5. **Réseau de contenu (services dockerisés)**

- Réseau des applications Dockerisées : `192.168.40.0/28`
- VLAN 1 sur les commutateurs : réservé à l'administration (Ansible).

6. **Liens privés entre opérateur et contenu/clients**

- IOU2 – IOU1 : `172.16.10.0/30`
- IOU6 – IOU7 : `172.16.10.4/30`
- IOU5 – IOU7 : `172.16.10.8/30`
- IOU5 – IOU8 : `172.16.20.0/30`
- Autre réseau privé pour gestion : `192.168.30.16/28`

7. **Sécurité et administration (SSH)**

Pour permettre l'administration via Ansible :

- SSH activé sur tous les routeurs du réseau via le réseau d'administration, avec l'utilisateur `cisco` et l'authentification par mot de passe.

Exemple de configuration SSH :

```
ip domain-name cisco
crypto key generate rsa general-keys modulu
s 2048
ip ssh version 2
ip ssh time-out 60
ip ssh authentication-retries 3
service password-encryption
enable password cisco
username cisco password cisco
line vty 0 4
login local
transport input ssh
```

Pour accéder à distance : `ssh cisco@192.168.30.X`

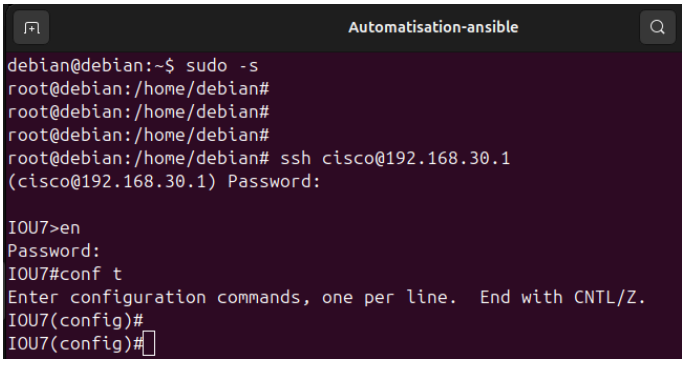


Tableau récapitulatif du plan d'adressage

Zone	Usage / Équipements	Plage d'adresses / IP	Masque	Remarques
Clients 1	Réseau interne, PC, softphone, loopback	192.168.10.0 – 192.168.10.7	/29	DHCP activé sur IOU7
Clients 2	Réseau interne, PC, softphone, loopback	192.168.20.0 – 192.168.20.7	/29	DHCP activé sur IOU8
Opérateur – Loopbacks	IOU2, IOU3, IOU4, IOU5, IOU6	10.10.2.2 /32 etc.	/32	Pour OSPF & identification
Liens opérateur	Liaisons point à point entre routeurs opérateur	10.10.10.0 – 10.10.10.27	/30	7 liens inter-routeurs
Multicast IPTV	Diffusion multicast (TV/IPTV)	230.0.0.0 – 230.0.0.7	/29	Classe D, pour le service IPTV
Réseau d'admin	Ansible, Switch1, liens d'administration	192.168.30.0 – 192.168.30.15	/28	Pour la gestion & SSH
Contenu/Docker	VM avec conteneurs (VoIP, VOD, IPTV, Web)	192.168.40.0 – 192.168.40.15	/28	Réseau applications/services
Privé intersites	Liens privés entre opérateur & clients/contenu	172.16.10.0 – 172.16.20.3	/30	4 liens privés point à point

Configuration des équipements réseau

1. **Affectation des adresses IP (par zone/équipement)**

   Je commence par affecter les adresses IP sur chaque interface des équipements selon le plan d'adressage prédéfini.

   Exemple (pour IOU2) :

   IOU2

```
interface Loopback0
 ip address 10.10.2.2 255.255.255.255
 ip pim sparse-mode
!
interface FastEthernet0/0
 ip address 192.168.30.3 255.255.255.240
 duplex full
!
interface GigabitEthernet1/0
 ip address 172.16.10.6 255.255.255.252
 ip pim sparse-mode
 negotiation auto
!
interface GigabitEthernet2/0
 ip address 10.10.10.1 255.255.255.252
 ip pim sparse-mode
 negotiation auto
!
interface GigabitEthernet3/0
 ip address 10.10.10.5 255.255.255.252
 ip pim sparse-mode
 negotiation auto
```

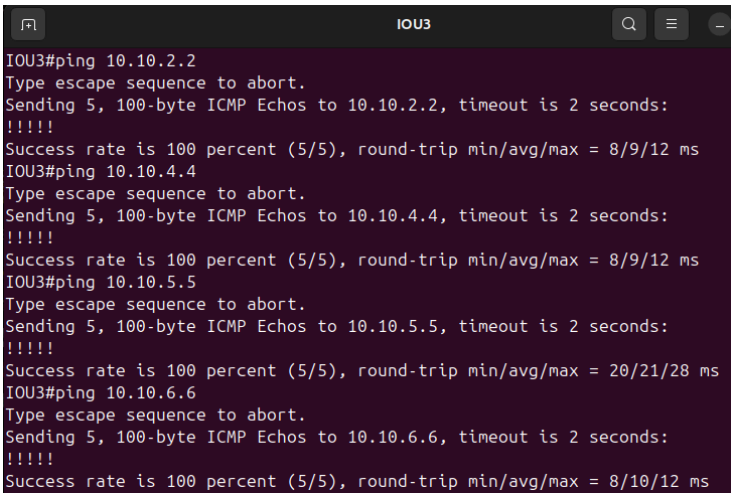
2. **Routage OSPF sur l'infra opérateur**

   Tous les routeurs de l'opérateur utilisent OSPF pour assurer le routage dynamique. Les loopbacks sont utilisées pour l'ID du routeur.

Exemple de config :

```
router ospf 1
router-id 10.10.3.3
network 10.10.10.0 0.0.0.255 area 0
network 10.10.2.2 0.0.0.0 area 0
```

Démonstration de ping - Le routeur IOU3 ping tous les réseaux loopback de l'infra opérateur.

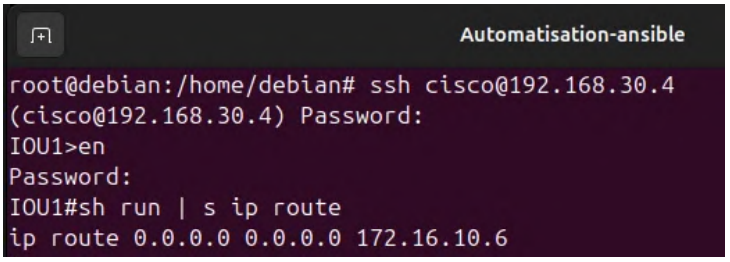


3. **Routage et config sur les réseaux de contenu et clients**

   Pour le **réseau de contenu** et les **réseaux clients**, j'ai choisi de mettre en place du **routage statique**. Cela simplifie la gestion car la topologie est simple et les routes ne changent pas souvent.

   Exemple de configuration de routage statique

   Sur un routeur du réseau de contenu (par exemple IOU1) :



   Sur un routeur client (exemple IOU7 pour Client 1) :

   Routage statique pour joindre le réseau de l'opérateur à partir du tunnel DMVPN mis en place.

```
Automatisation-ansible
root@debian:/home/debian# ssh cisco@192.168.30.1
(cisco@192.168.30.1) Password:
IOU7>en
Password:
Password:
IOU7#sh run | s ip route
ip route 0.0.0.0 0.0.0.0 192.168.200.1
```

J'ai fait le choix du routage statique sur ces réseaux parce que ça permet de garder une configuration simple et maîtrisée. Les routes sont fixées manuellement, ce qui évite toute ambiguïté sur le chemin des paquets.

4. **Multicast et IGMP**

   Pour permettre la diffusion de la TV sur IP (IPTV), j'ai activé le routage multicast sur les routeurs concernés.

```
Automatisation-ansible
IOU4#sh run | s ip multicast-routing
ip multicast-routing
```

On utilise PIM-SM (Protocol Independent Multicast – Sparse Mode) ainsi qu'IGMP pour la gestion des groupes multicast.

Voici la configuration PIM-SM sur toutes les interfaces participant au multicast :

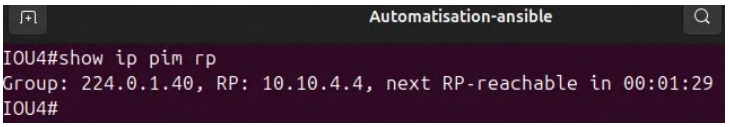
```
Automatisation-ansible
 negotiation auto
!
interface GigabitEthernet2/0
 ip address 10.10.10.17 255.255.255.252
 ip pim sparse-mode
 negotiation auto
!
interface GigabitEthernet3/0
 ip address 10.10.10.10 255.255.255.252
 ip pim sparse-mode
 negotiation auto
!
interface GigabitEthernet4/0
 ip address 10.10.10.21 255.255.255.252
 ip pim sparse-mode
 negotiation auto
```

Grâce à ces commandes, on peut vérifier l'état du routage multicast, la détection des voisins PIM, la présence des groupes IGMP, et s'assurer que les flux IPTV peuvent bien circuler sur tout le réseau :

```
IOU4#show ip pim neighbor
PIM Neighbor Table
Mode: B - Bidir Capable, DR - Designated Router, N - Default DR Priority,
      P - Proxy Capable, S - State Refresh Capable, G - GenID Capable
Neighbor          Interface                Uptime/Expires    Ver   DR
Address                                                            Prio/Mode
192.168.200.3     Tunnel10                 01:17:51/00:01:39 v2    1 / DR S P G
192.168.200.2     Tunnel10                 01:18:51/00:01:29 v2    1 / S P G
10.10.10.5        GigabitEthernet1/0       01:18:34/00:01:25 v2    1 / S P G
10.10.10.18       GigabitEthernet2/0       01:18:32/00:01:29 v2    1 / DR S P G
10.10.10.9        GigabitEthernet3/0       01:19:02/00:01:28 v2    1 / S P G
10.10.10.22       GigabitEthernet4/0       01:18:34/00:01:24 v2    1 / DR S P G
IOU4#show ip pim rp
Group: 230.0.0.1, RP: 10.10.4.4, next RP-reachable in 00:00:10
Group: 224.0.1.40, RP: 10.10.4.4, next RP-reachable in 00:01:28
```

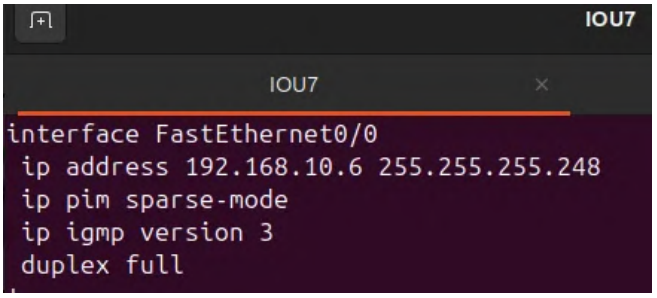
J'ai choisi le routeur IOU4 (loopback 10.10.4.4) comme RP pour le protocole PIM-SM. Tous les routeurs concernés par la diffusion multicast reçoivent cette configuration pour garantir la découverte correcte des flux multicast sur le réseau.

ip pim rp-address 10.10.4.4



Le routage multicast PIM-SM est activé sur toutes les interfaces participant à la distribution IPTV, y compris sur les interfaces Tunnel du hub et des spokes (clients).

Cependant, j'ai activé IGMP uniquement sur l'interface LAN du routeur de bordure côté client (spoke), là où les utilisateurs finaux rejoignent les groupes multicast (ex : VLAN ou interface Ethernet du client).



Pour assurer la diffusion des flux IPTV, j'ai activé le routage multicast sur toute l'infrastructure en utilisant PIM-Sparse Mode (PIM-SM). Toutes les interfaces qui participent à l'acheminement du trafic multicast, y compris les interfaces tunnel DMVPN et la loopback du routeur RP, sont configurées avec PIM sparse-mode. J'ai défini le routeur IOU4 (loopback 10.10.4.4) comme Rendezvous Point pour centraliser la gestion des groupes multicast. Côté clients, IGMP est activé sur les interfaces LAN pour permettre aux équipements de rejoindre dynamiquement les groupes IPTV. Des commandes de vérification comme **show ip pim neighbor**, **show ip pim rp** et **show ip igmp groups** confirment que la configuration fonctionne correctement.

Mise en place de la sécurité

Pour assurer la connectivité sécurisée entre les clients et l'opérateur, j'ai mis en place un tunnel DMVPN (Dynamic Multipoint VPN) avec un chiffrement IPsec. Cette solution permet aux routeurs de transporter à

la fois le trafic normal et les flux multicast de manière sécurisée, sans avoir à reconfigurer tout le réseau à chaque ajout de site.

- **Configuration sur le Hub**

  Sur le routeur Hub (ici IOU4), j'ai configuré l'interface tunnel multipoint DMVPN avec un profil IPsec pour le chiffrement, ainsi que les paramètres NHRP pour permettre l'enregistrement dynamique des spokes.

  Génération de la clé crypto et configuration de l'IPsec :

  IOU4

```
!
crypto isakmp policy 10
 authentication pre-share
crypto isakmp key cisco address 0.0.0.0
!
!
crypto ipsec transform-set cisco esp-aes esp-sha-hmac
 mode tunnel
!
crypto ipsec profile cisco
 set transform-set cisco
```

  Interface Tunnel (Hub) :

```
interface Tunnel10
 ip address 192.168.200.1 255.255.255.0
 no ip redirects
 ip pim sparse-mode
 ip nhrp authentication cisco
 ip nhrp map multicast dynamic
 ip nhrp network-id 1
 ip nhrp registration no-unique
 tunnel source GigabitEthernet2/0
 tunnel mode gre multipoint
 tunnel key 100
 tunnel protection ipsec profile cisco
```

  ip pim sparse-mode : permet de transporter le multicast sur le tunnel DMVPN.

  ip nhrp ... : paramètres DMVPN pour l'enregistrement dynamique et le transport du multicast.

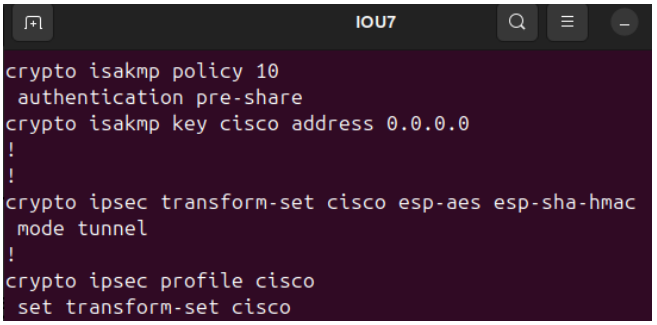
  tunnel protection ... : applique le chiffrement IPsec sur le tunnel.

  Cette configuration permet au Hub de gérer dynamiquement l'enregistrement des différents sites (spokes) et d'assurer le chiffrement de tous les échanges via IPsec. La présence de PIM sparse-mode permet aussi le transport sécurisé du multicast à travers le tunnel DMVPN.

- **Configuration sur le Spoke (client)**

  Sur chaque site client (spoke), le tunnel DMVPN est configuré pour s'enregistrer automatiquement auprès du Hub et permettre la circulation des flux multicast, le tout sécurisé par IPsec.

  Crypto et IPsec (identique au hub) :



Interface Tunnel (Spoke) et l'Interface LAN (vers clients) :

```
interface Tunnel10
 ip address 192.168.200.2 255.255.255.0
 no ip redirects
 ip pim sparse-mode
 ip nhrp authentication cisco
 ip nhrp map multicast dynamic
 ip nhrp map 192.168.200.1 10.10.10.17
 ip nhrp map multicast 10.10.10.17
 ip nhrp network-id 1
 ip nhrp nhs 192.168.200.1
 tunnel source GigabitEthernet4/0
 tunnel mode gre multipoint
 tunnel key 100
 tunnel protection ipsec profile cisco
!
interface FastEthernet0/0
 ip address 192.168.10.6 255.255.255.248
 ip pim sparse-mode
 ip igmp version 3
 duplex full
```

ip nhrp nhs 192.168.200.1 : indique l'adresse du Hub.

ip pim sparse-mode sur Tunnel10 et FastEthernet0/0 : permet le passage du multicast du tunnel jusqu'aux clients.

ip igmp version 3 : permet aux clients du LAN de s'abonner aux flux multicast.

J'ai configuré sur le routeur spoke (IOU7) le tunnel DMVPN pour s'enregistrer automatiquement auprès du Hub. Le multicast est activé pour transporter les flux IPTV, et IPsec chiffre l'ensemble du trafic. L'interface LAN (FastEthernet0/0) utilise IGMP pour permettre aux équipements clients de rejoindre dynamiquement les groupes IPTV.

Pour permettre le transport des flux multicast (ex : IPTV) à travers le tunnel DMVPN, il est nécessaire d'activer **PIM sparse-mode** sur les interfaces tunnel, aussi bien sur le Hub que sur les Spokes.

J'utilise également la configuration **NHRP** pour le support du multicast, en particulier les commandes :

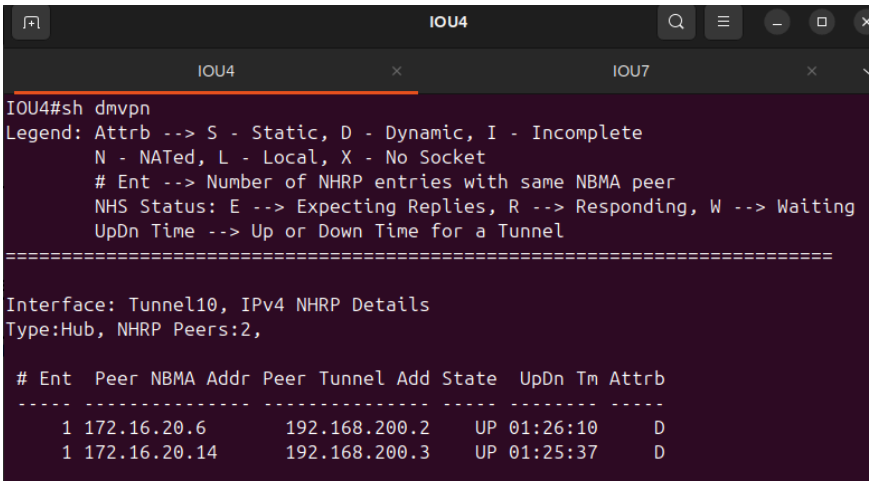
ip nhrp map multicast dynamic
ip nhrp map multicast @IP

Cela permet d'acheminer dynamiquement les paquets multicast à travers le tunnel GRE multipoint, en s'assurant que les groupes multicast sont bien propagés entre tous les sites connectés au VPN.

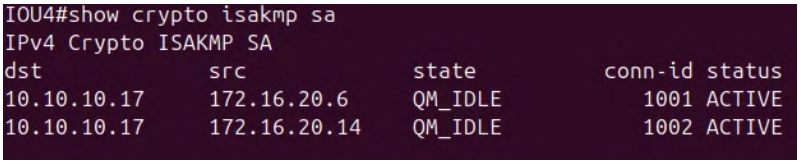
Grâce à cette configuration, les flux multicast sont correctement transportés d'un site à l'autre via le tunnel DMVPN, ce qui permet aux clients de recevoir les services IPTV quel que soit leur emplacement sur le réseau.

- **Résultats :**

  L'état des tunnels DMVPN et NHRP :



  L'état du tunnel IPsec :



  Les commandes de vérification montrent que le tunnel DMVPN est bien établi, que le chiffrement IPsec fonctionne, et que les flux multicast sont correctement transportés entre tous les sites. Tous les indicateurs sont au vert, ce qui confirme que la configuration est opérationnelle.

  → Lien vers les configurations des routeurs :

Automatisation de l'infrastructure

Pour simplifier le déploiement, la gestion et la maintenance du réseau, j'ai automatisé une partie de la configuration grâce à Ansible.

Ansible fonctionne à partir d'un simple fichier d'inventaire (où sont listés les équipements), et utilise des scripts appelés *playbooks* écrits en YAML. Ces playbooks décrivent les tâches à exécuter (comme configurer une interface ou activer un protocole), ce qui rend les déploiements rapides, reproductibles et faciles à mettre à jour à distance.

- **Exemple de tâches réalisées**

  J'ai d'abord créé un fichier d'inventaire qui liste tous les routeurs, avec leurs adresses IP, les paramètres d'accès (utilisateur, mot de passe) et les informations nécessaires à la connexion via Ansible.

→ Lien vers le fichier inventaire.yml

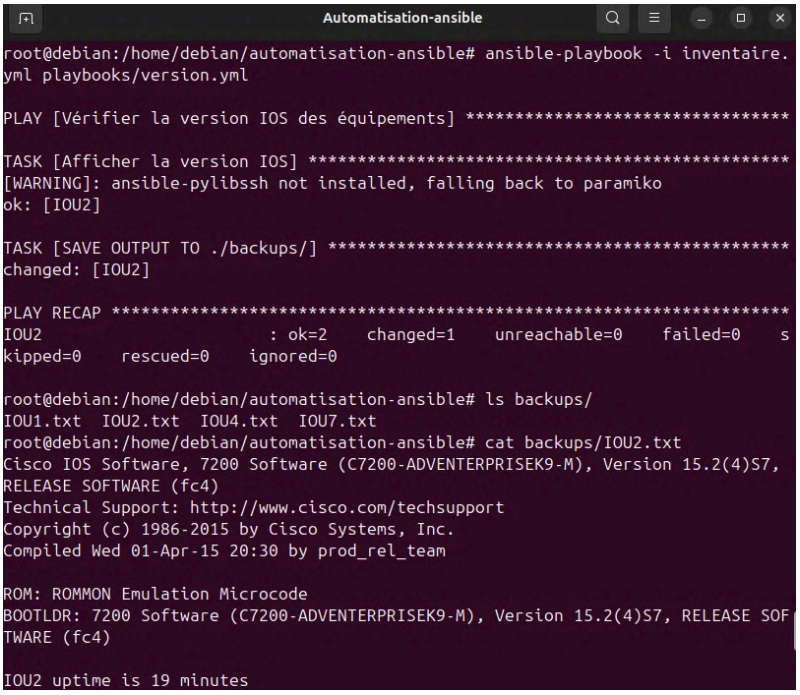
Ensuite, j'ai réalisé un premier playbook permettant de vérifier la version IOS d'un routeur (ici IOU2) et de sauvegarder automatiquement le résultat dans un fichier texte.

```
---
- name: Vérifier la version IOS des équipements
  hosts: IOU2
  gather_facts: no
  tasks:

    - name: Afficher la version IOS
      ios_command:
        commands: show version
      register: version_output

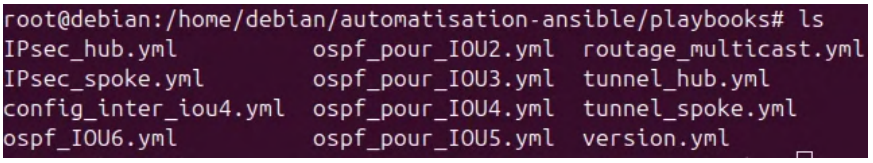
    - name: SAVE OUTPUT TO ./backups/
      copy:
        content: "{{ version_output.stdout[0] }}"
        dest: "backups/{{ inventory_hostname }}.txt"
```

Et en résultat, j'ai la version de l'équipement, dans un fichier texte, classé dans le dossier "backups".



- **Autres exemples des configurations que j'ai réalisées avec Ansible :**

  j'ai créé une série de playbooks Ansible, chacun dédié à une tâche précise sur l'infrastructure et j'ai pu configurer rapidement et de façon homogène.



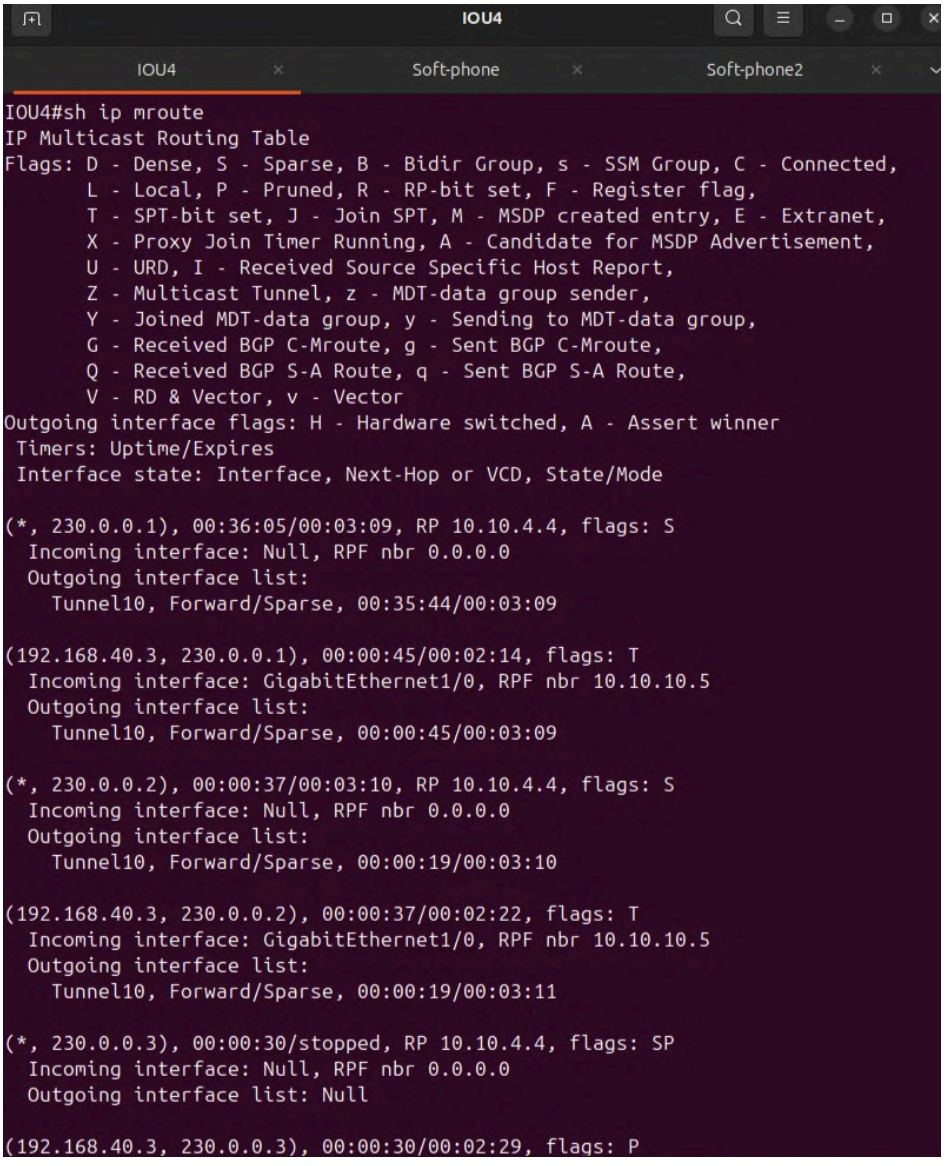
→ Lien vers les playbooks

L'automatisation avec Ansible m'a simplifié la vie : plus besoin de refaire les mêmes manipulations à la main, moins de risques d'oublier une commande, et tout se fait beaucoup plus vite. Ça m'a permis de rester organisé et d'avoir une config propre partout.

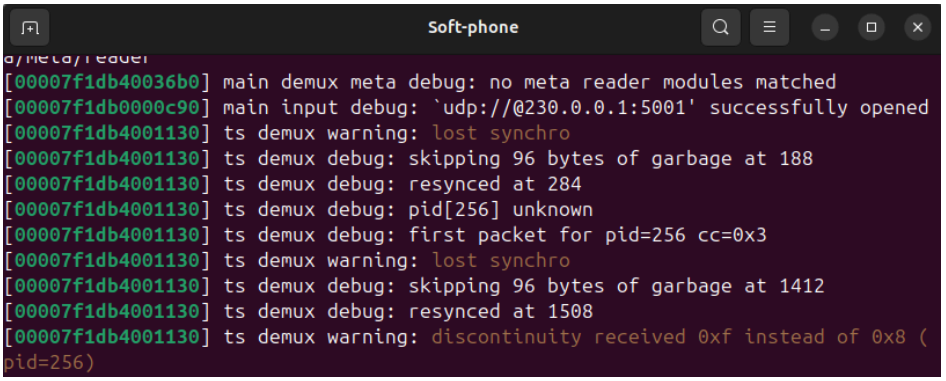
Résultats de l'infrastructure réseau

À la fin du déploiement, toutes les vérifications montrent que l'infrastructure fonctionne comme prévu :

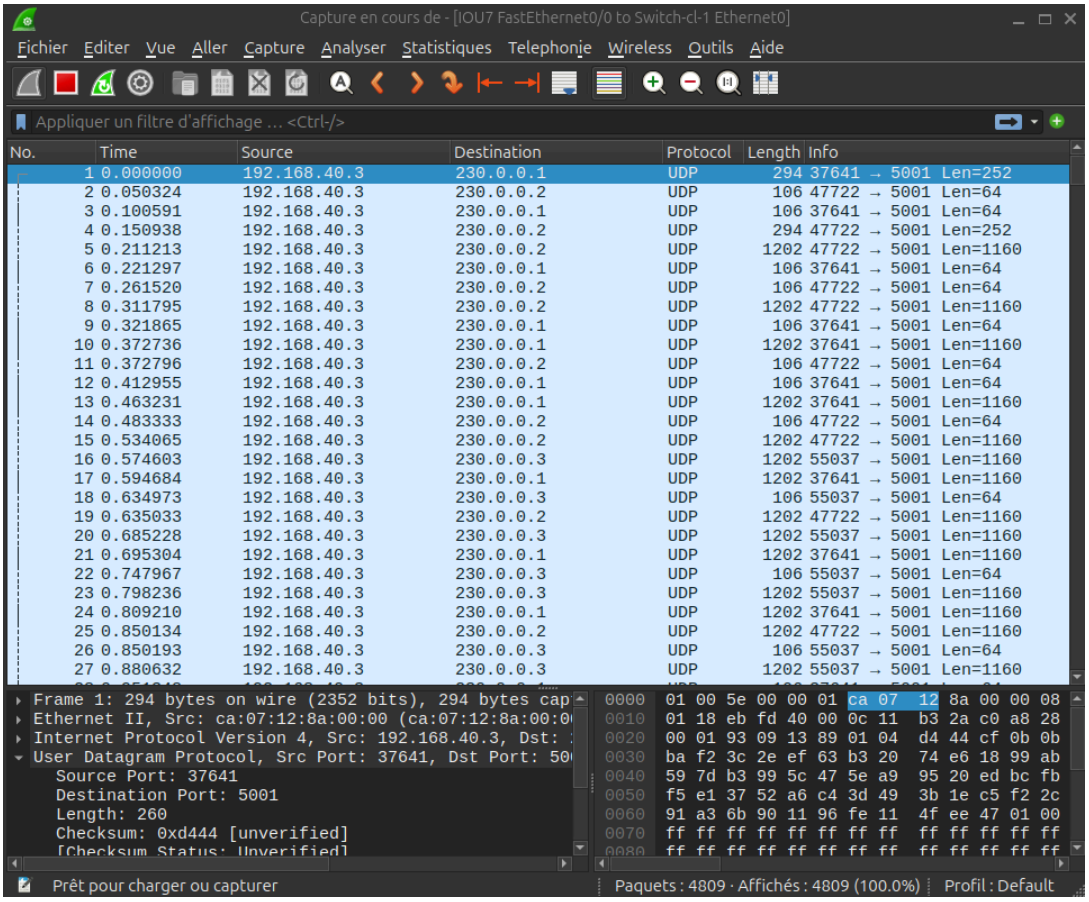
- **Table de routage multicast du hub (IOU4)** : la commande **show ip mroute** affiche bien la réception et la propagation des flux multicast sur les interfaces concernées, prouvant que le routage multicast est opérationnel au cœur du réseau.



- **Lecture des flux sur le client (VLC)** : en lançant le client VLC avec la commande : **vlc -vvv udp://@230.0.0.1:5001 (Sur le serveur qui envoie l'audio : ffmpeg -re -i test_audio.mp3 -f mpegts udp://230.0.0.1:5001)**, on voit que le client rejoint et reçoit effectivement le flux multicast, ce qui valide le bon fonctionnement du multicast jusqu'aux équipements terminaux.



- **Analyse Wireshark** : la capture réseau sur l'interface client montre le passage des trames UDP à destination des adresses multicast, confirmant la bonne distribution des paquets sur tout le chemin du réseau, du serveur jusqu'au client.



On voit clairement que le flux circule du serveur jusqu'au client, et que **l'infrastructure répond au cahier des charges**.

Déploiement des services réseaux dockerisés

Dans cette partie, on présente la mise en place des différents services réseaux (VoIP, VOD, IPTV, Web) à l'aide de Docker, directement sur une machine virtuelle dédiée. Pour chaque service, on explique : le but du service, la préparation de l'environnement Docker, le contenu des fichiers principaux (Dockerfile, docker-compose.yml, fichiers HTML/CSS...), le processus de déploiement, et la méthode d'accès ou de

test du service. On déploiera chaque services sur des ports différents mais en utilisant la même IP, celle de la machine.

1. Service VOD :

Objectif :

Le but de ce service est de mettre en place la Vidéo à la Demande (VOD), c'est-à-dire un site web permettant de visionner différentes vidéos directement depuis un navigateur. Nous avons utilisé Docker pour simplifier le déploiement et garantir que notre serveur web fonctionne de manière cohérente sur n'importe quel système.

Préparation de l'environnement :

- Mise à jour des paquets :
- Avant toute installation, nous mettons à jour la liste des paquets de notre système :

```
apt update
```

- Installation de Docker :
- Docker nous permet de créer et de gérer des conteneurs. Nous installons donc **Docker** avec la commande suivante :

```
apt install docker.io
```

Structure :

Nous avons créé un dossier `vod/` qui contiendra tous les fichiers nécessaires :

```
vod/
├── Dockerfile
├── docker-compose.yml
├── index.html
├── style.css
├── bresil.mp4
├── sdm.mp4
├── turc.mp4
```

Téléchargement des vidéos :

Nous avons téléchargé et ajouté dans le dossier `vod/` les vidéos que nous voulons mettre à disposition ici :

- `bresil.mp4`
- `sdm.mp4`
- `turc.mp4`

Construction de l'image Docker :

Dockerfile :

Le Dockerfile sert à définir l'image Docker que nous allons utiliser. Ici, nous utilisons l'image officielle du serveur Apache (httpd), qui est un serveur web.

```
FROM httpd:2.4
COPY . /usr/local/apache2/htdocs/
```

Explication :

- `FROM httpd:2.4` : nous utilisons l'image Apache HTTP Server version 2.4.
- `COPY . /usr/local/apache2/htdocs/` : on copie nos fichiers (html, css, vidéos) dans le répertoire de base du serveur web.

Utilisation de Docker Compose :

docker-compose.yml :

Nous utilisons Docker Compose pour automatiser le déploiement du conteneur :

```
version: '3'

services:
  vod:
    build: .
    container_name: vod
    ports:
      - "8080:80"
    restart: unless-stopped
```

Explication :

- `build: .` : construction de l'image à partir du Dockerfile.
- `container_name: vod` : nom du conteneur.
- `ports: "8080:80"` : le port **80** du conteneur est mappé sur le port **8080** de l'hôte.
- `restart: unless-stopped` : le conteneur redémarre automatiquement sauf si on l'arrête manuellement.

Fichiers du site :

index.html :

Player

GNU nano 7.2 index.html

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Vidéo à la demande</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>🎬 Vidéo à la demande</h1>

  <div class="container">
    <div class="video-card">
      <video controls>
        <source src="bresil.mp4" type="video/mp4">
      </video>
      <p>Musique brésilienne</p>
    </div>

    <div class="video-card">
      <video controls>
        <source src="sdm.mp4" type="video/mp4">
      </video>
      <p>SDM - Prince de la calle</p>
    </div>

    <div class="video-card">
      <video controls>
        <source src="turc.mp4" type="video/mp4">
      </video>
      <p>Musique Turc</p>
    </div>
  </div>
</body>
</html>
```

Style.css :

```
body {
  margin: 0;
  padding: 0;
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  background-color: #cca9dd;
  color: #333;
}

h1 {
  text-align: center;
  padding: 2rem 1rem 1rem;
  color: #3e3e3e;
  font-size: 2.5rem;
  margin-bottom: 0;
}

.container {
  max-width: 1000px;
  margin: auto;
  padding: 1rem;
  display: flex;
  flex-wrap: wrap;
  justify-content: center;
  gap: 1.5rem;
}

.video-card {
  background-color: #ffffff;
  border-radius: 12px;
  box-shadow: 0 2px 15px rgba(0, 0, 0, 0.08);
  padding: 1rem;
  width: 280px;
  transition: transform 0.2s ease-in-out;
}
```

```css
.video-card:hover {
  transform: translateY(-4px);
}

video {
  width: 100%;
  height: auto;
  border-radius: 8px;
  outline: none;
}

.video-card p {
  margin-top: 0.6rem;
  font-weight: 500;
  color: #555;
  text-align: center;
}
```

Déploiement :

Construction de l'image :

Nous construisons l'image Docker avec :

```
docker build -t vod .
```

Lancement du conteneur :

Ensuite, nous lançons le conteneur :

```
docker run -d --name vod -p 8080:80 vod
```

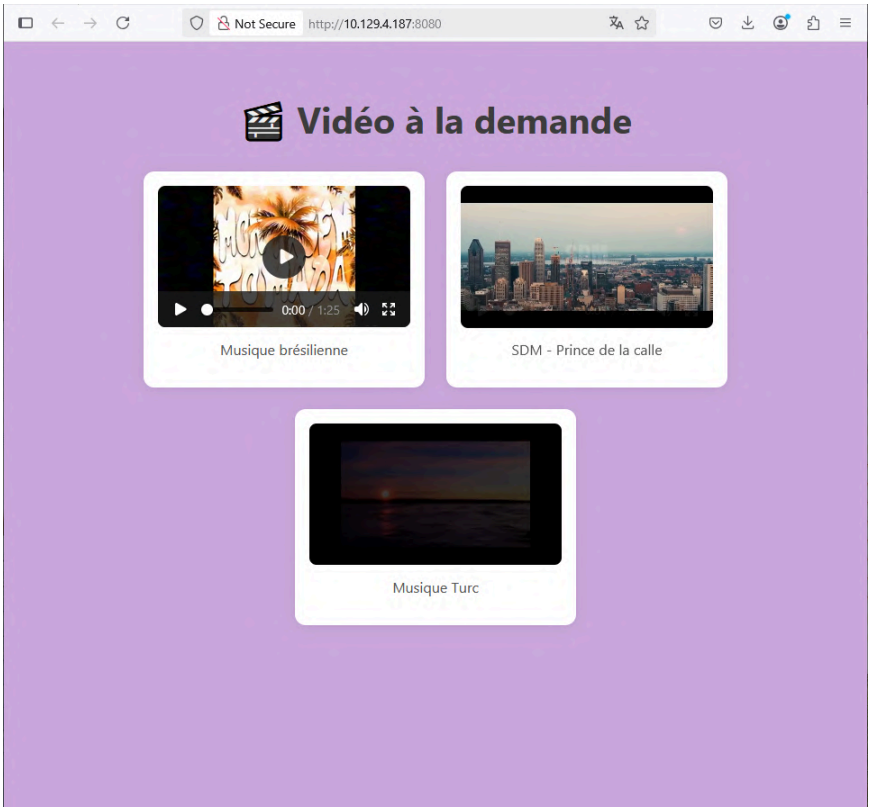
- `d` : mode détaché
- `-name vod` : nom du conteneur
- `p 8080:80` : mappage des ports

Accès au site :

Enfin, pour accéder au site, on utilise l'adresse IP de notre VM suivie du port 8080 dans un navigateur :

```
http://10.129.4.187:8080
```

Résultat :



2. Service IPTV :

Cette partie du projet a pour objectif de mettre en place un service IPTV (Internet Protocol Television), qui permet à un utilisateur de regarder des chaînes de télévision directement depuis un navigateur web. Grâce à une interface développée en HTML/CSS, l'utilisateur peut sélectionner et lire une chaîne en direct.

Structure :

```
iptv/
├── Dockerfile
├── index.html
├── style.css
```

Dockerfile (identique au projet VOD) :

Le Dockerfile est simple et permet de servir la page web via Apache (conteneur `httpd`).

```
FROM httpd:2.4
COPY . /usr/local/apache2/htdocs/
```

Contenu HTML :

L'objectif est de fournir une interface utilisateur pour choisir une chaîne via un menu déroulant et lire le flux sélectionné dans un lecteur vidéo.

```html
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>IPTV</title>
  <script src="https://cdn.jsdelivr.net/npm/hls.js@latest"></script>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <header>
    <h1>📺 IPTV - Sélectionne ta chaîne</h1>
  </header>

  <div class="channel-select-container">
    <select id="channelSelect" onchange="playSelectedChannel()">
      <option disabled selected>--- Choisir une chaîne ---</option>
    </select>
  </div>

  <video id="video" controls autoplay></video>

  <script>
    let channelMap = {};

    async function loadChannels() {
      const listUrl = 'https://iptv-org.github.io/iptv/countries/fr.m3u';
      const response = await fetch(listUrl);
      const text = await response.text();

      const lines = text.split('\n');
      const select = document.getElementById('channelSelect');

      for (let i = 0; i < lines.length; i++) {
        if (lines[i].startsWith('#EXTINF')) {
          const name = lines[i].split(',')[1];
          const url = lines[i + 1];
          channelMap[name] = url;
        }
```

```
      }

      const sortedNames = Object.keys(channelMap).sort();

      for (const name of sortedNames) {
        const option = document.createElement('option');
        option.value = name;
        option.textContent = name;
        select.appendChild(option);
      }
    }

    function playSelectedChannel() {
      const name = document.getElementById('channelSelect').value;
      const url = channelMap[name];
      playChannel(url);
    }

    function playChannel(url) {
      const video = document.getElementById('video');
      if (Hls.isSupported()) {
        if (window.hls) window.hls.destroy();
        const hls = new Hls();
        hls.loadSource(url);
        hls.attachMedia(video);
        window.hls = hls;
      } else if (video.canPlayType('application/vnd.apple.mpegurl')) {
        video.src = url;
      } else {
        alert("Votre navigateur ne supporte pas la lecture HLS.");
      }
    }

    window.onload = loadChannels;
  </script>

</body>
</html>
```

Explication du code :

Chargement de HLS.js :

```
<script src="https://cdn.jsdelivr.net/npm/hls.js@latest"></script>
```

Cette bibliothèque permet de lire des flux HLS (HTTP Live Streaming), qui ne sont pas pris en charge nativement par tous les navigateurs.

Menu de sélection :

```
<select id="channelSelect" onchange="playSelectedChannel()">
```

Ce menu est automatiquement rempli avec les chaînes disponibles dans le fichier `.m3u`.

Script JavaScript :

Le script effectue les étapes suivantes :

1. Télécharge le fichier `.m3u` depuis GitHub avec toutes les chaines :

```
const listUrl = 'https://iptv-org.github.io/iptv/countries/fr.m3u';
```

2. Analyse chaque ligne pour détecter les noms des chaînes et leurs URL.
3. Alimente dynamiquement le `<select>` avec ces chaînes triées par ordre alphabétique.
4. Lorsqu'une chaîne est sélectionnée, elle est lue dans un élément `<video>` via HLS.js :

```
hls.loadSource(url);
hls.attachMedia(video);
```

Si HLS.js n'est pas supporté, le script tente de lire la vidéo via les capacités natives du navigateur.

CSS :



```css
body {
  margin: 0;
  padding: 0;
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  background-color: #cca9dd;
  color: #333;
  text-align: center;
}

h1 {
  font-size: 2.7rem;
  color: #2c3e50;
}

.channel-select-container {
  padding: 20px;
  text-align: center;
}

select {
  padding: 10px 14px;
  font-size: 16px;
  width: 320px;
  border-radius: 8px;
  border: 1px solid #ccc;
  box-shadow: 0 1px 6px rgba(0, 0, 0, 0.05);
  background-color: #fff;
  font-weight: 500;
  color: #333;
}

video {
  width: 960px;
  height: 540px;
  display: block;
  margin: 30px auto;
  border-radius: 10px;
  border: 2px solid #ddd;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.08);
```



```css
@media screen and (max-width: 1024px) {
  video {
    width: 90%;
    height: auto;
  }

  select {
    width: 90%;
  }
}
```

Déploiement :

Construction de l'image :

```
docker build -t iptv .
```

Lancement du conteneur :

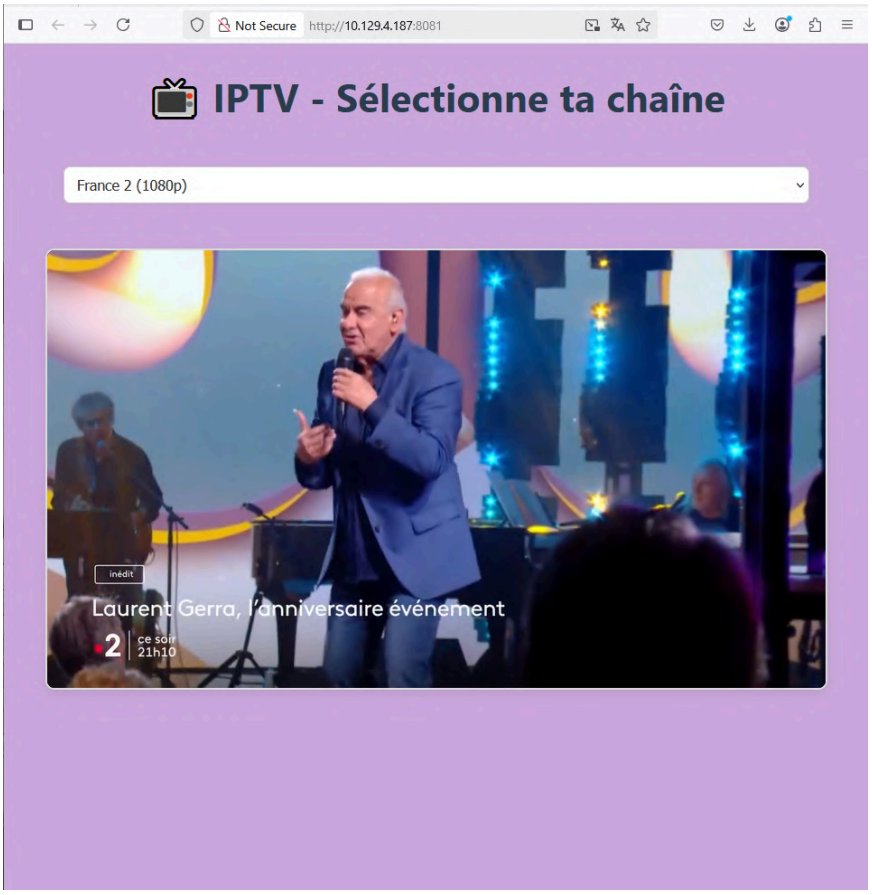
```
docker run -d --name iptv -p 8081:80 iptv
```

Accès au site :

Enfin, pour accéder au site, on utilise l'adresse IP de notre VM suivie du port 8081 dans un navigateur :

```
http://10.129.4.187:8081
```

Notre page s'affiche bien avec l'écran principale et le menu déroulant avec toutes les chaines en direct a regarder. On en compte plus d'une centaine.



3. Service Web :

Ce service web d'accueil sert de page centrale permettant à l'utilisateur d'accéder facilement aux deux services précédemment mis en place : VOD et IPTV

L'interface prend la forme d'une page HTML simple, contenant deux boutons cliquables redirigeant vers les services correspondants.

Structure du projet :

```
accueil-tv/
├── Dockerfile
├── index.html
├── style.css
├── docker-compose.yml
```

Dockerfile :

Le `Dockerfile` utilisé est identique à celui des services IPTV et VOD :

```
FROM httpd:2.4
COPY . /usr/local/apache2/htdocs/
```

Ce fichier permet d'exposer la page via un serveur HTTP Apache dans un conteneur Docker.

Docker Compose :

Le fichier `docker-compose.yml` permet de faciliter le déploiement :

```
version: '3'

services:
  vod:
    build: .
    container_name: accueil
    ports:
      - "8000:80"
    restart: unless-stopped
```

Page HTML :

Le but est d'afficher une page avec deux boutons redirigeant vers :

- Le service VOD (`http://10.129.4.187:8080`)
- Le service IPTV (`http://10.129.4.187:8081`)

```
 GNU nano 7.2                                  index.html
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Accueil TV</title>
  <link rel="stylesheet" href="accueil.css">
</head>
<body class="accueil">
  <h1>🎬 Bienvenue sur l'accueil TV</h1>
  <div class="button-group">
    <a class="button" href="http://10.129.4.187:8080" target="_blank">🎞️Accéder à la VOD</a>
    <a class="button" href="http://10.129.4.187:8081" target="_blank">📺 Accéder à l'IPTV</a>
  </div>
</body>
</html>
```

Le CSS :



```
 GNU nano 7.2                                accueil.css
body.accueil {
  margin: 0;
  padding: 0;
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  background-color: #cca9dd;
  color: #333;
  text-align: center;
  padding-top: 100px;
}

body.accueil h1 {
  font-size: 2.7rem;
  margin-bottom: 60px;
  color: #2c3e50;
}


.button
{

 padding: 8px;
 font: 25px calibri;
 color: white;
 background-color: black;
 border: none;
 border-radius: 20px;
 transition: all 0.3s ease 0s;
 cursor: pointer;
 box-shadow: 0px 5px 7.5px rgba(0, 0, 255, 0.54);
}
.button:hover
{
 background-color: #ffffff;
 box-shadow: 0px 7.5px 10px rgba(0, 0, 255, 0.54);
 color: #000000;
 transform: translateY(-7px);
}

.button:active
{
 transform: translateY(-1px);
}
```

Déploiement :

Construction de l'image :

```
docker build -t accueil .
```

Exécution du conteneur :

```
docker run -d --name accueil -p 8000:80 accueil
```

Accès au service :

Une fois le conteneur lancé, on accède à l'accueil via un navigateur :

```
http://10.129.4.187:8000
```

L'interface s'affiche correctement, avec deux boutons menant directement aux services :

- IPTV : `http://10.129.4.187:8081`
- VOD : `http://10.129.4.187:8080`



Le service d'accueil complète parfaitement l'infrastructure en offrant une interface centralisée et ergonomique. Il simplifie l'accès pour l'utilisateur.

4. Service VoIP :

Dans le cadre de ce projet, il a fallut faire la mise en place d'un service VoIP en utilisant Asterisk en tant que serveur SIP, et Linphone comme client pour passer et recevoir des appels. L'objectif était d'obtenir une communication audio entre deux utilisateurs connectés via un serveur Asterisk, dans un environnement virtualisé.

J'ai commencé par installer Asterisk, et me suis orienté vers ses trois fichiers de configuration principaux : `pjsip_wizard.conf`, pour déclarer les utilisateurs SIP, `extensions.conf`, pour définir les règles d'appel, et `pjsip.conf` pour le transport. Voici la configuration que j'ai mise en place dans `pjsip_wizard.conf` :

```
[1001]
type=wizard
transport=transport-udp
accepts_auth=yes
accepts_registrations=yes
inbound_auth/type=userpass
inbound_auth/username=1001
inbound_auth/password=1234
aor/max_contacts=1
aor/remove_existing=1
endpoint/direct_media=no
endpoint/allow=!all,alaw
endpoint/message_context=Messages
endpoint/callerid=1
```

```
[1002]
type=wizard
transport=transport-udp
accepts_auth=yes
accepts_registrations=yes
inbound_auth/type=userpass
inbound_auth/username=1002
inbound_auth/password=1234
aor/max_contacts=1
aor/remove_existing=1
endpoint/direct_media=no
endpoint/allow=!all,alaw
endpoint/message_context=Messages
endpoint/callerid=2
```

Chaque utilisateur (1001 et 1002) est ici identifié avec le mot de passe (1234).

Ensuite, dans `extensions.conf`, j'ai ajouté les lignes ci-dessous permettant à chaque poste d'appeler l'autre :

```
[default]
exten => 1,1,Dial(PJSIP/1001)
exten => 2,1,Dial(PJSIP/1002)
```

Ces règles permettent à un poste SIP d'appeler l'autre en composant simplement le numéro correspondant.

Puis dans `pjsip.conf`, j'active la pile de signalisation classique SIP/UDP en décommentant les lignes suivantes :

```
[transport-udp]
type=transport
protocol=udp
bind=0.0.0.0
```

Une fois cette configuration effectuée, j'ai tenté de connecter mes clients Linphone à l'adresse IP de la VM, en utilisant les identifiants déclarés. L'objectif était qu'ils s'enregistrent auprès du serveur et soient prêts à initier des appels.

Malheureusement, je n'ai pas réussi à faire fonctionner le service de manière opérationnelle. Linphone ne parvenait pas à s'enregistrer, et affichait des messages d'échec de connexion.

J'ai pensé a un problème de communication réseau, notamment au niveau des ports. Asterisk utilise par défaut le port UDP 5060 pour les connexions SIP, et une plage de ports pour les flux RTP. Si ces ports ne sont pas ouverts ou correctement redirigés dans l'environnement réseau, les clients ne peuvent pas dialoguer avec le serveur.

J'ai tenté de trouver le problème en consultant les logs d'Asterisk avec la commande :

```
asterisk -rvvvv
```

Cela m'a permis de voir certaines tentatives de connexion, mais pas suffisamment d'informations pour identifier clairement le problème. Après réflexion je pense que `sngrep` aurait été utile pour visualiser le trafic SIP, mais je n'y ai pas pensé sur le coup.

Malgré l'absence de résultat fonctionnel, cette tentative m'a permis de mieux comprendre la VoIP basé sur Asterisk. J'ai déclaré des comptes utilisateurs, conçu un plan de numérotation, et configuré un client SIP comme Linphone. Ce fut aussi une initiation concrète aux problématiques réseau que pose la VoIP, notamment la gestion des ports.

Si je devais refaire cette partie du projet, je commencerais par des tests en réseau local simple, sans passer par Docker, afin d'isoler les problèmes de configuration pure des problèmes réseau. J'aurais aussi utilisé `sngrep` ou `Wireshark` pour suivre les échanges entre clients et serveur.

En conclusion, même si le service VoIP n'a pas pu aboutir à une communication réussie, cette expérience m'a permis de poser les bases de compréhension d'un système VoIP, et de mieux appréhender les couches réseau et applicatives qui le composent.

Gestion de projet

Pour ce projet, on a travaillé en binôme du début à la fin. La plupart du temps, chacun avançait de son côté, souvent le soir à la maison ou pendant nos moments libres.

Pour nous organiser et documenter toutes les étapes, on a utilisé un outil en ligne très pratique qui s'appelle **Notion**. Voici le lien de l'espace Notion qu'on a utilisé pour centraliser et gérer toutes les informations de notre projet : SAE4.ROM.01 - Déployer une infrastructure opérateur et ses services clients

Conclusion

En conclusion, ce projet nous a permis de mettre en place toute une infrastructure réseau complète, du routage au déploiement des services multimédias avec Docker. On a dû réfléchir, tester, parfois galérer, mais au final tout fonctionne comme prévu : les clients accèdent bien aux services (VOD, IPTV, VoIP, Web), le multicast passe, et la sécurité est assurée avec le VPN.
On a aussi découvert la force de l'automatisation avec Ansible, qui nous a vraiment fait gagner du temps et évité les erreurs.
Travailler à deux, en s'organisant entre la maison, l'IUT et en échangeant sur les réseaux, ça nous a bien aidés à avancer et à ne rien lâcher.

Au final, on est fiers d'avoir mené ce projet jusqu'au bout et d'avoir appris autant, que ce soit sur la technique ou sur la gestion de projet.

Nous sommes fiers du résultat, car tout fonctionne bien et répond au cahier des charges. Nous avons beaucoup appris et ça nous a donné encore plus envie de continuer à travailler sur des projets réseau comme celui-ci. Nous nous sentons plus confiant dans nos compétences, et nous sommes contents du chemin parcouru. *Merci de nous avoir lu !*